today will be simply hosted interlinked html files. Most websites today are dynamic applications that utilize databases and programming scripts that dynamically generate the contents you see on any webpage, and they have been doing so for quite some time. However now an increasing number of websites feature code which runs on the client computer as well. As browsers get faster, the web applications are getting more powerful as well.

Additionally websites are relying increasingly on content that the users themselves contribute. Combined with the dynamic server-side and client-side environments, this exposes a large number of security concerns as malicious users have more vectors of attack.

We shall be looking at some common flaws in web applications that allow web application hackers to wreck varying amounts of havoc. Web application authors are well aware of such vulnerabilities, and as they are found they get patched, however with the growing complexity of web applications mistakes are bound to occur.

The Open Web Application Security Project (OWASP) is a non-profit organization that aims to improve the security of software. To aid this effort, they provide many tools, and documentation for the same. We will be looking at the "OWASP Top 10 for 2010" list, which is their list of the top ten security risks affecting web applications. To demonstrate these attacks we will be using WebGoat, and intentionally insecure web application that the organization has made available. WebGoat – a pun on scapegoat – intends to showcase these vulnerabilities and expects you to attack them to learn more.

We will take a look at each of the ten vulnerabilities from the list, of which a key few will be explored in greater detail. The text within quotes is taken directly from the original report with no "hacking".

3.2.2 Injection

"Injection flaws, such as SQL, OS, and LDAP injection, occur when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing unauthorized data."

What this means is that in any application which processes data supplied by the user in an interpreter, care must be taken to ensure that an attacker cannot trick the interpreter into running any code they he / she wants.

This interpreter could be anything, however usually and commonly we hear of SQL Injection attacks, in which attackers take advantage of any part of an application that passes on user-supplied data unprocessed – or poorly processed – to the SQL database.

Due to the ubiquity of SQL servers, we shall look exclusively at SQL Injection attacks, however they are not the sole target of Injection attacks, other interpreters such as XPath, LDAP, etc. can also be exposed the same